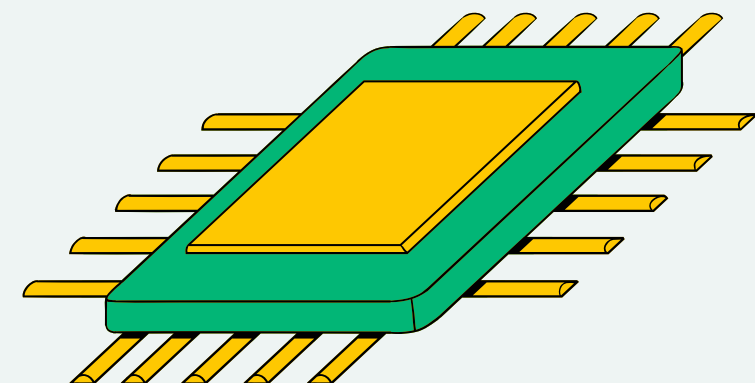


IMAGE-TO-IMAGE TRANSLATION WITH CONDITIONAL ADVERSARIAL NETWORKS

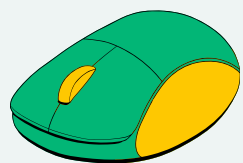
PRESENTED BY:

**KUNAL GOURV
(230002036)**



THE PROBLEM-

Image-to-image translation means changing one type of image into another—like turning a sketch into a photo or adding color to a black-and-white picture. Earlier, each task needed its own special method, which made things slow and harder to manage.



IMPORTANCE OF TRANSLATION

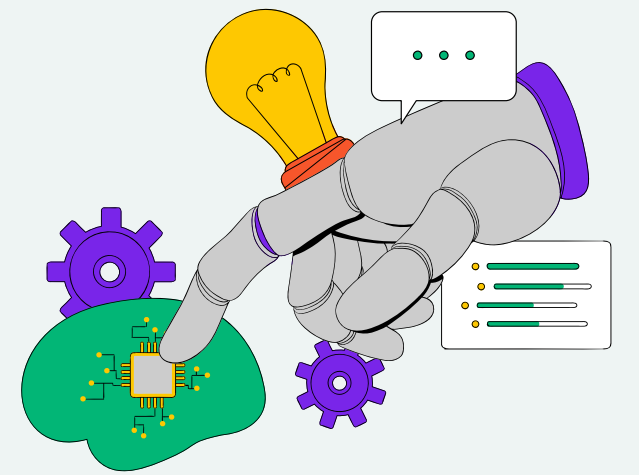
Image-to-image translation is important because it does tasks automatically that used to need a lot of manual work. Tools like Pix2Pix make the process simple and can be used for things like creating art or making images look more realistic. This helps save time and effort in creative work.

40



REAL WORLD EXAMPLE-

Image-to-image translation is used in real life for things like turning building sketches into realistic pictures or making simple maps look like detailed satellite images. This shows how useful the technology is in many fields like design, city planning, and entertainment.



NOVELTY

WHAT IS NEW?

CHALLENGES IN TRADITIONAL MODELS

MANUAL ALGORITHM-

Traditional methods necessitate the creation of specific algorithms for each unique task, which can limit versatility. For instance, tasks like coloring or edge detection require distinct setups, making scaling resource-intensive and complex.



TIME-CONSUMING PROCESSES

The requirement for manual design and algorithm development leads to a significantly prolonged workflow. Each specialized model not only necessitates complex programming but also involves extensive testing and fine-tuning, consuming both time and resources. It's a key driver of AI applications, including natural language processing, image recognition, and recommendation systems.

LIMITATIONS OF SPECIALIZED MODELS

Specialized models often fall short in adaptability, restricting their use to predefined tasks. This exclusivity hinders the overall efficiency and innovation potential in image tasks, as new algorithms must be written for every new application.



OVERVIEW OF PIX2PIX



Goal of Pix2Pix-

The primary goal of Pix2Pix is to create a unified model that can perform various image-to-image translation tasks, such as converting sketches to photorealistic images. This innovation eliminates the need for multiple specialized algorithms, thereby streamlining the translation process.



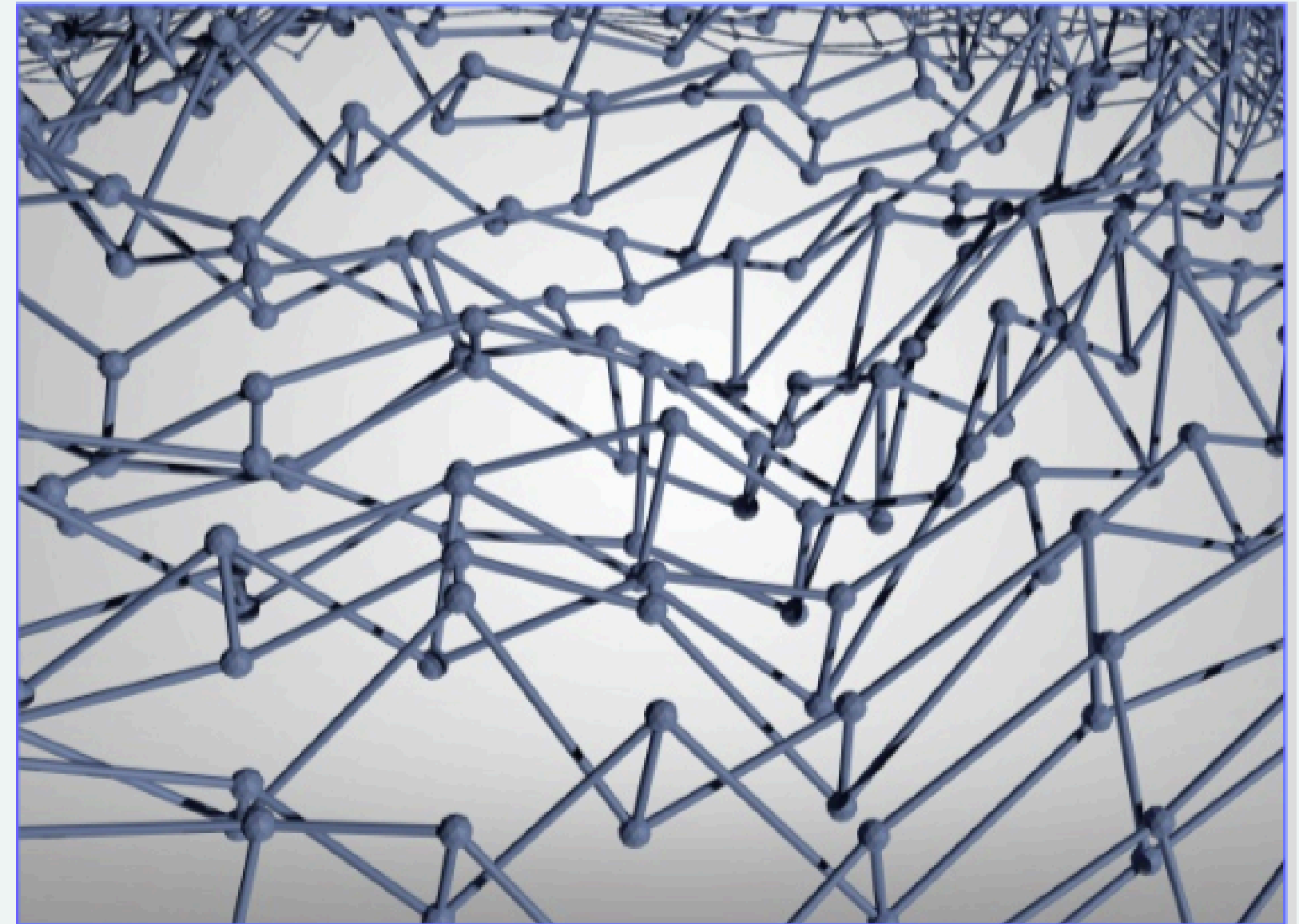
CONDITIONAL GENERATIVE ADVERSARIAL NETWORKS-

Pix2Pix employs a Conditional GAN architecture, comprising a Generator and a Discriminator. The Generator creates realistic images while the Discriminator differentiates between real and generated images, facilitating a competitive training process that improves output quality.



Structure of the Model

The Pix2Pix model is structured with a U-Net as the Generator, enabling it to learn complex features through skip connections. The PatchGAN Discriminator enhances image quality by focusing on local patches, ensuring that fine details are retained in the final output.



GENERATOR NETWORK (U-NET):

The Generator is used to translate an input image (like a satellite photo) into a realistic output image (like a map).

Encoder (Downsampling layers):

These layers compress the input image into a small, rich feature map by progressively reducing the height and width but increasing the number of channels.

→ This helps the model understand the global structure (like roads, shapes).

Bottleneck (Middle layer):

This is the most compressed version of the image — where all important information is squeezed into a small space.

Decoder (Upsampling layers):

These layers expand the compressed feature map back to the original image size, but now the output is generated to look like a real photo.

Skip Connections:

Skip connections directly pass details from the encoder to the decoder.

→ This helps preserve fine details like edges, borders, small objects that would otherwise get lost during compression.

PatchGAN Discriminator

The Discriminator's job is to judge whether an output image is real (from the dataset) or fake (from the generator).

A normal GAN discriminator would look at the whole image (like 256×256 pixels) and give one answer: real or fake.

In PatchGAN, the discriminator looks at small patches (like 70×70 pixel windows) one at a time..

- It slides over the image in small patches.
- For each patch, it decides whether that patch is real or fake.
- Instead of one decision, it outputs a grid of decisions (like a 30×30 map of real/fake).

Why use patches?

- Small patches focus on textures.
- It helps the model learn local details like sharp edges, bricks on buildings, leaves on trees.
- Training becomes faster because it doesn't have to analyze the whole image at once.
- Fully convolutional:
- You can apply the model to bigger images too — because there are no fixed-size dense layers.
- outputs a grid of decisions (like a 30×30 map of real/fake).

```
def forward(self, a, b):  
    x = torch.cat([a, b], dim=1)  
    return self.model(x)
```


DESCRIPTION AND UNIQUE FEATURE

DESCRIPTION OF THE CODE-

It follows the original Pix2Pix paper method exactly, step-by-step:

- Goal:

Given an input image (like satellite photo), generate a corresponding output image (like a MAP).

- How?

Using a Conditional GAN:

- A Generator G tries to produce realistic fake images $G(x)$.
- A Discriminator D tries to tell apart real images (x,y) and fake ones $(x,G(x))$



STEPS HAPPENING IN CODE-

datasets link-<https://www.kaggle.com/datasets/alincijov/pix2pix-maps>

Install libraries:

Install PyTorch, torchvision, matplotlib, tqdm
(progress bar):

Dataset setup:

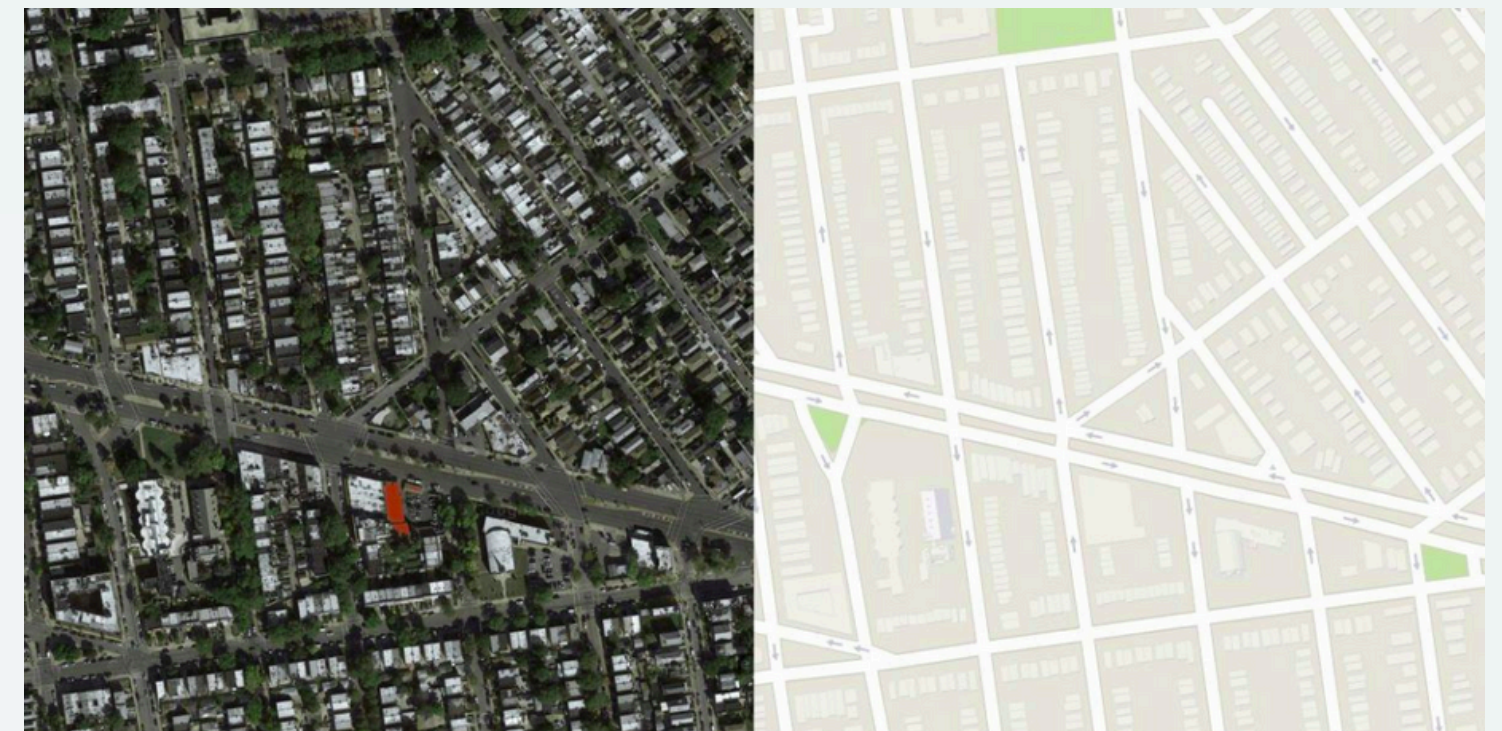
Load side-by-side images (input | target).

Cut them into two halves — left side = input (A),
right side = target (B).

Normalize images to range $[-1,1]$ to help training.

```
def __getitem__(self, idx):  
    img = Image.open(self.files[idx]).convert("RGB")  
    w, h = img.size  
    w2 = w // 2  
  
    img_A = img.crop((0, 0, w2, h))      # left half: input  
    img_B = img.crop((w2, 0, w, h))      # right half: target
```

EXAMPLE-



Training Loop:

First, update Discriminator:

- Train it to tell real from fake.

Then, update Generator:

- Train it to fool the Discriminator (make outputs realistic).
- Also minimize L1 distance (pixel differences) to ground truth.

Loss Function- (two losses -GAN loss +pixel loss)

The objective of a conditional GAN can be expressed as

$$\mathcal{L}_{cGAN}(G, D) = \mathbb{E}_{x,y} [\log D(x, y)] + \mathbb{E}_{x,z} [\log(1 - D(x, G(x, z)))], \quad (1)$$

$$\mathcal{L}_{L1}(G) = \mathbb{E}_{x,y,z} [\|y - G(x, z)\|_1].$$

The loss we use is:

$$\min_G \max_D (\mathcal{L}_{cGAN}(G, D) + 100 \mathcal{L}_1(G))$$

$\mathcal{L}_{cGAN}$ = fool the discriminator.

$\mathcal{L}_1$ = pixel-wise difference between output and real image (keep it close).

Training Objectives & Hyperparams

GAN loss: fool D

Combined loss:

L1 loss: match pixels

$$\mathcal{L} = \mathcal{L}_{cGAN} + 100\mathcal{L}_1$$

```
D = PatchDiscriminator().to(device)
opt_G = optim.Adam(G.parameters(), lr=2e-4, betas=(0.5, 0.999))
opt_D = optim.Adam(D.parameters(), lr=2e-4, betas=(0.5, 0.999))
criterion_GAN = nn.BCEWithLogitsLoss()
criterion_L1 = nn.L1Loss()
```


Data & Implementation Details

Satellite view->Map

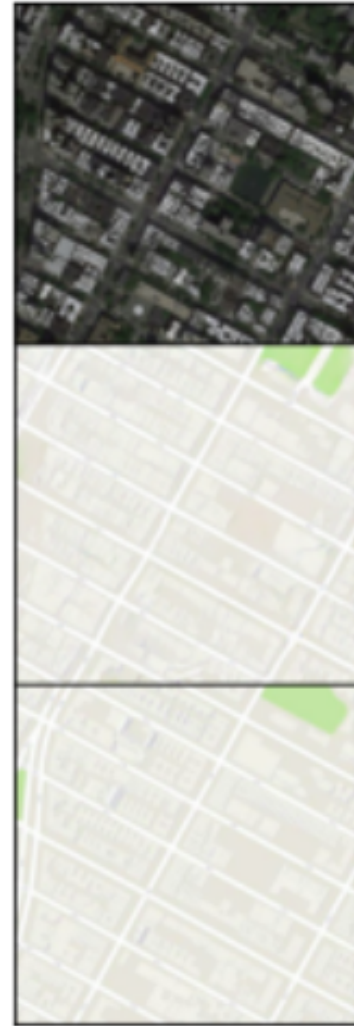


Results-

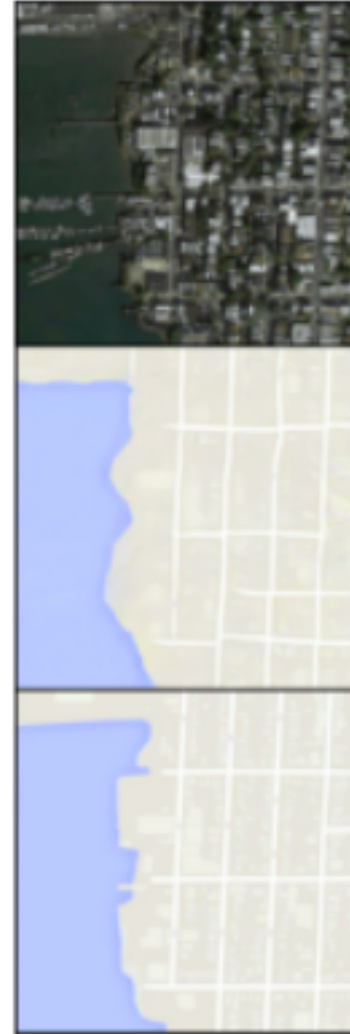
Epoch 1



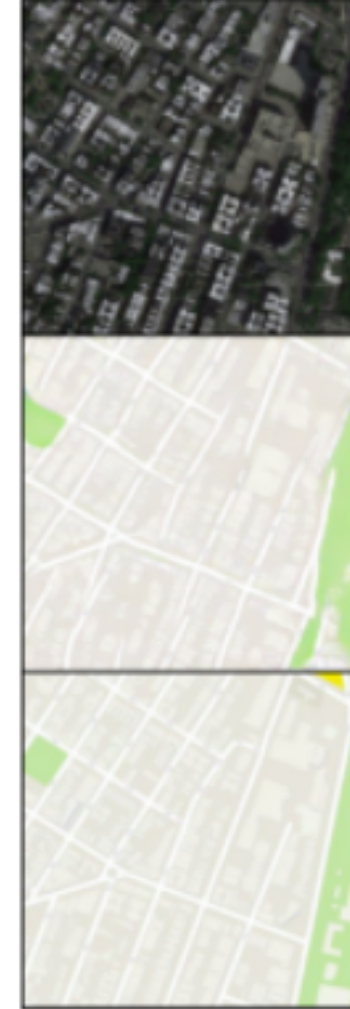
Epoch 10



Epoch 29



Epoch 82



Epoch 100



CONCLUSION-

In the beginning, the Generator (G) is still learning how to map the input to output.

It does not know what real photos look like yet!

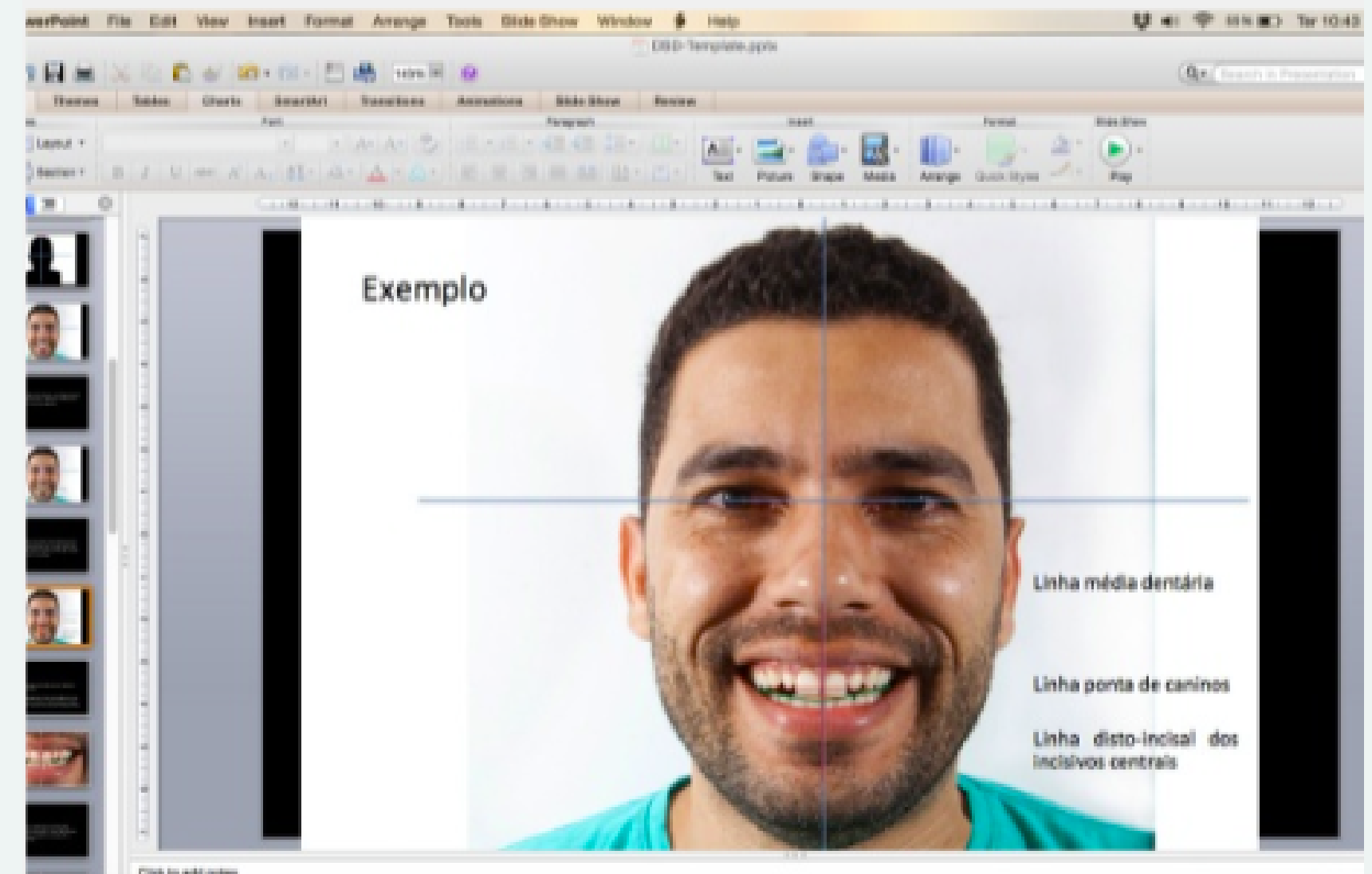
It only knows "something" should come out – so the outputs are very blurry and rough.

In the late stages of training, around 80 to 100 epochs, the generator becomes highly skilled at producing near-photorealistic outputs. By this time, it has successfully learned two important things: first, to match the overall structure of the target image using the L1 loss; second, to create realistic small textures like trees, windows, and roads using the adversarial GAN loss. As a result, the images generated are almost indistinguishable from real photos, showing sharp edges, correct shapes, and natural colors. The training stabilizes at this point because both the generator and discriminator have become strong opponents, constantly improving against each other. This leads to high-quality, realistic images that maintain both global consistency and fine local details.

Innovations and Contributions

One Model for Various Tasks

Before Pix2Pix, each image manipulation task required unique models, which was inefficient. Pix2Pix allows a single model to handle diverse translations, such as colorization and edge-to-photo transformations, dramatically simplifying the workflow and reducing time investment.





Learned Loss Mechanism

Earlier, models needed manually designed loss functions to check how good an image was. Pix2Pix improved this by letting the model learn by itself what a "good" image looks like, using a helper model called a discriminator. This makes training easier and helps the model work better without needing lots of manual adjustments.

Challenges

1. Training is Very Slow
2. Train the model 2 or 3 times to get good result.
3. Presence of limited GPUs
4. Got blurry output at the beginning

PAPER-

Image-to-Image Translation with Conditional Adversarial Networks

Phillip Isola Jun-Yan Zhu Tinghui Zhou Alexei A. Efros

Berkeley AI Research (BAIR) Laboratory, UC Berkeley
`{isola, junyanz, tinghuiz, efros}@eecs.berkeley.edu`

link- <https://arxiv.org/abs/1611.07004>

Comments: Website: [this https URL](https://arxiv.org/abs/1611.07004), CVPR 2017

Subjects: **Computer Vision and Pattern Recognition (cs.CV)**

Cite as: [arXiv:1611.07004](https://arxiv.org/abs/1611.07004) **[cs.CV]**

(or [arXiv:1611.07004v3](https://arxiv.org/abs/1611.07004v3) **[cs.CV]** for this version)

<https://doi.org/10.48550/arXiv.1611.07004> 

THANK YOU